

Case Study

Üretim Performans Takip Uygulaması

Mühendislik Aday Değerlendirme Case Study'si

Başvurduğunuz pozisyon için yetkinliğinizi değerlendirmek üzere hazırlanmış bir teknik görevdir. Doküman ~10 dakikada okunacak şekilde tasarlanmıştır. Lütfen kodlamaya başlamadan önce baştan sona okuyun.

1. Bağlam

Bir otomotiv yan sanayi firmasında çalışan üretim mühendisliği ekibi, enjeksiyon kalıplama (injection molding) hattındaki performansı vardiya bazında takip ediyor. MES (Manufacturing Execution System) sistemi her vardiya sonunda otomatik olarak .csv formatında üretim raporları üretiyor.

Bugüne kadar bu raporlar Excel'de manuel olarak inceleniyor; bu süreç:

- Zaman alıcı — mühendis başına haftalık 4–6 saat
- Hata yapmaya açık — kopya-yapıştır ile yapılan filtreleme, formül kayması
- Geriye dönük analiz zor — geçmiş 6 ayın trendine bakmak için her seferinde dosyalar birleştiriliyor

Üretim direktörü, ekipten hat performansını ve OEE (Overall Equipment Effectiveness) metriklerini gösteren web tabanlı bir uygulama geliştirilmesini istedi. Uygulama, doğrulanmış veriyi merkezi sisteme REST API üzerinden de gönderebilmelidir. Sizin göreviniz bu uygulamanın MVP'sini (Minimum Viable Product) tek başınıza geliştirmek.

2. Görev Tanımı (Yüksek Seviye)

Aşağıdaki yetenekleri sağlayan tam-stack bir web uygulaması geliştireceksiniz:

- CSV formatında gelen ham üretim verisini sisteme yüklemek (import)
- Veriyi SQLite veritabanında saklamak
- Filtreleme arayüzü üzerinden tarih, vardiya, iş istasyonu, ürün gibi boyutlarda veriyi sorgulamak
- OEE, üretim miktarı, fire oranı, duruş analizi gibi metrikleri raporlamak
- **Yüklenen verideki kalite sorunlarını tespit etmek, raporlamak ve gerektiğinde düzeltmek**
- **Doğrulanmış (temiz) veriyi REST API üzerinden hedef sisteme göndermek**

Kritik: 5. ve 6. madde bu görevin en önemli iki parçasıdır. Verinin "temiz" geldiğini varsaymayın. Gerçek MES verisi gibi, içinde veri kalitesi sorunları ve mantıksal tutarsızlıklar bulunmaktadır. Bu sorunları nasıl tespit ettiğiniz ve sadece güvenilir veriyi nasıl ileri sisteme aktardığınız değerlendirmenin merkezindedir.

3. Teknik Gereksinimler

Katman	Tercih
Frontend	React (tercih edilen) veya Python tabanlı UI (Streamlit, Dash kabul edilebilir)
Backend	Python (FastAPI veya Flask önerilir)
Veritabanı	SQLite (zorunlu)
Veri Formatı	CSV import
API Entegrasyonu	HTTP/REST (requests, httpx veya muadili)
Versiyon Kontrol	Git + GitHub (zorunlu)

Kütüphane seçimi serbestir (pandas, SQLAlchemy, pydantic, vb.). Ancak seçimlerinizin gerekçesini README'de belirtmeniz gerekir.

4. Sağlanan Veri

4.1 Dosya

production_data.csv — 2.117 satır, 18 kolon, 3 haftalık üretim raporu (5–25 Kasım 2025)

4.2 Veri Sözlüğü

#	Kolon	Tip	Açıklama
1	record_id	int	Kayıt tekil numarası
2	Tarih	date	Üretim günü
3	İş Emri No	string	İş emri tekil numarası (302 ile başlayan 10 haneli sayı, örn. 3025678325)
4	İş Merkezi No	string	İş merkezi kodu
5	İşmerkezi Adı	string	İş merkezi tanımı
6	İş İstasyon Adı	string	Makine kodu (örn. IMM-2700-3)
7	Stok Adı	string	Üretilen parça/ürün
8	Vardiya	int	Vardiya numarası (1, 2 veya 3)
9	A (Kullanılabilirlik)	float	Availability — yüzde (0–100)
10	P (Performans)	float	Performance — yüzde
11	Q (Kalite)	float	Quality — yüzde (0–100)
12	OEE	float	Overall Equipment Effectiveness — yüzde
13	Çalışma Süresi	float	Makine çalışma dakikası
14	Duruş Süresi	float	Toplam duruş dakikası
15	Planlı Duruş Süresi	float	Planlı duruş dakikası
16	Plansız Duruş Süresi	float	Plansız duruş dakikası

17	Üretilen Miktar	int	Toplam üretim adedi
18	Hatalı Üretilen Miktar	int	Fire/scrap adedi

4.3 OEE Formülü (referans)

OEE = A × P × Q / 10000 (yüzde formatında)

OEE endüstri standardı bir metriktir:

- A (Availability) = Çalışma Süresi / (Çalışma Süresi + Plansız Duruş Süresi)
- P (Performance) = İdeal hıza karşı gerçekleşen hız
- Q (Quality) = (Üretilen – Hatalı) / Üretilen

5. Fonksiyonel Gereksinimler (Detaylı)

5.1 Veri İçe Aktarma (Import) Arayüzü

Zorunlu:

- Kullanıcı bir CSV dosyası seçebilmeli (drag-and-drop veya file picker)
- Yükleme öncesi önizleme gösterilmeli (ilk 5–10 satır)
- Yükleme sırasında ilerleme durumu kullanıcıya bildirilmeli
- Yükleme sonrası özet rapor gösterilmeli:
 - Toplam satır sayısı
 - Başarıyla import edilen satır sayısı
 - Reddedilen / şüpheli satır sayısı ve sebebi
 - Verideki kalite sorunlarının dökümü
- Aynı dosya tekrar yüklendiğinde duplicate kontrolü yapılmalı

Tercih edilen:

- Birden fazla CSV dosyasını birleştirerek yükleyebilme

5.2 Filtreleme & Sorgulama Ekranı

Aşağıdaki boyutlarda çoklu filtre desteklenmeli:

- Tarih aralığı (başlangıç–bitiş)
- Vardiya (1, 2, 3 — çoklu seçim)
- İş İstasyonu (multi-select)
- Stok Adı / Ürün
- OEE değer aralığı (slider)
- Sadece sorunlu kayıtları göster (toggle)

Filtreleme anlık çalışmalı (sayfa yenilenmeden) ve filtrelenen kayıtlar CSV olarak dışa aktarılabilmesi.

5.3 Üretim Raporu / Dashboard

En az aşağıdaki görselleri içermeli:

- OEE trendi (günlük/haftalık çizgi grafik)
- Vardiya bazlı performans karşılaştırma
- İş istasyonu bazlı OEE sıralaması
- Fire oranı (Quality) dağılımı
- Özet KPI kartları: Ortalama OEE, Toplam Üretim, Toplam Fire, Toplam Duruş

5.4 Veri Validasyonu ve Kalite Raporu (Kritik)

Bu bölüm değerlendirmede en yüksek ağırlığa sahiptir.

Beklenen davranış:

- CSV yüklendiğinde sistem otomatik olarak veri kalitesi kontrolü çalıştırmalı
- Tespit edilen her hatalı/şüpheli kayıt için bir validation report üretilmeli
- Bu rapor şunları içermeli:
 - record_id
 - Hata tipi (sınıflandırılmış)
 - Hangi alan(lar)da hata var
 - Önerilen aksiyon: reddet / uyar / düzelt
- Kullanıcı:
 - Şüpheli kayıtları toplu görüntüleyebilmeli
 - Manuel düzeltme yapabilmeli VEYA reddedebilmeli
 - Düzeltme geçmişini görebilmeli (audit trail — tercih edilen)

Hangi hataları arayacağınızı kendiniz tasarlamalısınız. Üretim mantığını, OEE matematiğini ve veri tutarlılığını düşünerek validasyon kuralları yazın. İpucu olarak şu kategorileri düşünün:

- Eksik / boş veri — hangi alanlar zorunlu, hangileri opsiyonel?
- Aralık dışı değerler — yüzdelerin sınırları, miktarların işareti
- Tutarsız ilişkiler — fire ile üretim ilişkisi, süre toplamları, OEE bileşenleri
- Duplicate kayıtlar — hangi alanlar bir kaydı tekil yapar?
- Format tutarsızlıkları — tarih, metin alanları
- Domain mantığı — fiziksel olarak imkânsız değerler

İpucu: Ne kadar çok hata tipi tespit ederseniz o kadar yüksek puan alırsınız. Ancak her tespit ettiğiniz hatayı gerekçesiyle ve örnek kayıtlarla belgelemelisiniz. Yanlış pozitifler puan kaybettirir.

5.5 API Entegrasyonu — Hedef Sisteme Veri Gönderme

Validasyondan geçen ve "temiz" olarak işaretlenen kayıtlar, harici bir REST API üzerinden hedef sisteme gönderilmelidir. Gönderim her gün, 3 vardiya için ayrı ayrı gerçekleştirilmelidir (Vardiya 1 – Sabah / Vardiya 2 – Öğle / Vardiya 3 – Gece).

Zorunlu:

- Validate edilmiş (hatasız) kayıtlar, kullanıcı bir "Gönder" / "Senkronize Et" butonuna bastığında REST API'ye POST edilmeli
- İstek body'si JSON formatında olmalı
- Kimlik doğrulama: X-Production-Key header ile API key gönderilmeli
- Gönderim sonucu kullanıcıya bildirilmeli:
 - Başarılı gönderim sayısı
 - Başarısız (HTTP 4xx/5xx dönen) kayıt sayısı ve hata mesajları
- Yeniden deneme (retry) mekanizması — başarısız kayıtlar için
- Gönderim idempotent olmalı: aynı kayıt iki defa gönderilirse hedef sistemde duplicate oluşmamalı

Tercih edilen:

- Toplu gönderim (batch) — tek tek değil, gruplar halinde POST edilsin
- Hedef sistem loglanması — hangi kayıt ne zaman, hangi cevap ile gönderildi

- Async / background gönderim (kullanıcı UI'da beklemesin)

Endpoint ve Alan Bilgileri:

JSON Alanı	Açıklama
oe_value (float)	Ekipman verimliliği — yüzde, 0.0–100.0 arası
machine_count (integer)	Vardiyada aktif makine sayısı — 1–1000 arası
shift (integer)	Vardiya: 1 (Sabah), 2 (Öğle), 3 (Gece)
total_production_units (integer)	Toplam üretim adedi — 1–1.000.000 arası
production_date (string)	Üretim tarihi — YYYY-MM-DD formatı, gelecek tarih kabul edilmez

Örnek İstek:

```
POST /api/v1/submit X-Production-Key: <your-key> Content-Type: application/json {
  "machine_count": 12, "total_production_units": 4500, "oe_value": 87.3, "shift":
  1, "production_date": "2026-05-07" }
```

Beklenen Başarılı Yanıt:

```
HTTP 200 OK { "success": true, "submission_id": 42, "candidate_name": "Firstname
Lastname", "message": "Data recorded successfully. ID #42.", "submitted_at":
"2026-05-07T08:30:00" }
```

Hata Kodları:

HTTP Kodu	Anlam
401	Eksik veya geçersiz API key
422	Validasyon hatası — yanıtta detail alanını inceleyin
429	Rate limit aşıldı — 1 dakika bekleyin
413	İstek gövdesi 10 KB sınırını aştı

API endpoint URL ve API key bilgileri **.env** dosyasından okunmalıdır. Bu bilgileri kod içine gömmeyin. **.env** dosyası **.gitignore** ile repo'dan hariç tutulmalı; yerine **.env.example** paylaşılmalıdır.

 Tam API dokümantasyonu için: <http://89.252.189.91:8983/docs-guide> — Detaylı kural ve kısıtlamalar için lütfen bu adresi inceleyiniz.

6. Değerlendirme Kriterleri

Kategori	Ağırlık	Bakılacak Şeyler
Veri Validasyon Kapsamı	%25	Kaç farklı hata tipi yakalandı, gerekçeleri ne kadar net, yanlış pozitif var mı
API Entegrasyonu	%15	Auth, idempotency, retry, error handling, secret yönetimi
Fonksiyonel Tamamlık	%15	Import, filtreleme, raporlama çalışıyor mu
Kod Kalitesi	%15	Modülerlik, isimlendirme, testler, error handling
UI/UX	%10	Kullanılabilirlik, kullanıcıya hata bildirimleri ne kadar anlaşılır
Mimari ve Teknik Tercihler	%10	Frontend-backend ayrımı, DB şeması, API tasarımı

Dokümantasyon	%5	README, kurulum talimatları, validasyon kurallarının açıklanması
AI Kullanım Şeffaflığı	%5	Bölüm 8'e bakınız

7. Teslimat

7.1 GitHub Repository

Public veya invite-based private repo. Repo adı: **[isimsoyisim]-uretim-takip-case**

 Teslimat: Tamamlandıktan sonra **tunahan.ozturk@magna.com** adresine e-posta ile iletiniz.

README.md zorunlu içerikler:

- Projenin amacı (kısaca)
- Hızlı kurulum talimatları — komut komut, adım adım
- Hızlı çalıştırma talimatları
- Uygulama ekran görüntüleri — Dashboard, Import, Validasyon raporu ve API gönderim ekranı
- Tespit ettiğiniz hata tiplerinin listesi ve örnekleri
- API entegrasyon akışı
- Kullanılan kütüphaneler ve seçim gerekçeleri
- Yapamadığınız / vakit yetmeyen kısımlar (dürüst olun, puan kaybettirmez)
- Daha fazla zaman olsaydı neyi geliştirirdiniz?

.env.example dosyası zorunludur (gerçek .env repo'da olmamalı).

7.2 Çalıştırma

Repoyu klonlayan biri **3 komuttan az** ile uygulamayı ayağa kaldırabilmelidir. Test verisi (production_data.csv) repo içinde **data/** klasöründe bulunmalıdır.

```
# Kurulum git clone https://github.com/<kullanici>/<repo-adi>.git cd <repo-adi> cp .env.example .env # Backend cd backend && pip install -r requirements.txt uvicorn main:app --reload # Frontend (ayrı terminal) cd frontend && npm install && npm run dev # Uygulama: http://localhost:5173
```

8. Yapay Zeka (AI) Kullanım Politikası

AI araçlarını (ChatGPT, Claude, Copilot vb.) kullanabilirsiniz. Bunu yasaklamıyoruz çünkü gerçek hayatta da kullanıyorsunuz. Ancak şu kuralları zorunlu olarak uygulayacaksınız:

- Chat geçmişlerinizi paylaşacaksınız. Repo içinde ai_usage/ klasörü oluşturup tüm AI görüşmelerini ekran görüntüsü, paylaşım linki veya text dump olarak ekleyin.
- Hangi prompt için hangi AI'ı kullandığınızı belirtin. Örn: 01_database_schema.md, 02_validation_rules.md
- AI çıktısını body olduğu gibi kopyalamadan önce anladığınızdan emin olun. Mülakat sunumunda kodun her satırını açıklamanızı bekleyeceğiz.
- AI'a yaptırdığınız iş transparan olduğu sürece sorun yok. Saklama girişimi ise sürecin sonunu getirir.

9. Süre

2 iş günü

Bu görev gerçek bir iş yükü olarak tasarlanmıştır. Kıymetli bulduğumuz şey "kusursuz bitirmek" değil, gerçekçi sürede karar veren ve önceliklendiren mühendis görmektir. Yetiştirememeniz halinde ne yapamadığınızı README'de dürüstçe belirtmeniz, abartılı vaatlerden çok daha iyi görünür.

10. Bonus / İdeal Çözüm

Aşağıdakiler zorunlu değil ama gerçek bir mühendislik bakışı varsa kendiliğinden ortaya çıkarlar:

- Validasyon kurallarının kullanıcı tarafından (UI'dan) düzenlenebilir olması
- Birim testleri (özellikle validasyon mantığı için)
- Performans: 100K+ satırlık veriyi de kabul edebilen import akışı
- Pre-existing anomalileri yakalamak ve "sistemik bir sorun mu, tekil bir kayıt mı" olarak ayırmak
- Backend için API dokümantasyonu (OpenAPI/Swagger)
- Data lineage: Her validation hatası için CSV satır izlenebilirliği
- İndirilebilir validation report (Excel/PDF)
- API gönderim geçmişinin (history) UI'da gösterilmesi
- Circuit breaker / exponential backoff — API tarafında uzun süreli hata durumunda

11. İkinci Aşama: Sunum

İlk değerlendirmeden geçen adaylar 30–45 dakikalık teknik sunum yapacaklardır. Sunum şu başlıkları içermelidir:

- Mimari kararlarınız — neden bu stack, neden bu DB şeması
- Validasyon yaklaşımınız — hangi hataları nasıl yakaladınız, nasıl sınıflandırdınız
- API entegrasyon akışı — auth, retry, idempotency stratejiniz
- En zor karşılaştığınız problem ve nasıl çözdüğünüz
- Canlı demo — bilgisayarınızdan
- Q&A — kodun belirli yerlerini açıklamamız istenecek

12. Sıkça Sorulan Sorular

S: Bütün hataları yakalamak zorunda mıyım?

C: Hayır. Yakaladığınız hataları net biçimde belgelendirdiğiniz sürece kısmi puan alırsınız. Hiç bulamayanlar düşük puan alır.

S: Tasarımcı değilim, UI ne kadar şık olmalı?

C: Tailwind veya hazır bir component library (MUI, Ant Design, shadcn/ui) tamamen kabul edilebilir. Pixel-perfect istemiyoruz, kullanılabilir istiyoruz.

S: Tüm verinin doğru olduğunu mu varsayayım, yoksa şüphecilik mi?

C: Şüphecilik. Bu görevin asıl test ettiği şey budur.

S: Tarih formatları, ondalık ayrıçlar tutarsız olabilir mi?

C: Olabilir. Gerçek MES verisi bunu gösterir.

S: Kütüphane veya AI seçimi konusunda kısıt var mı?

C: Hayır, ama tercih ettiklerinizi gerekçelendirin.

S: Verideki kayıtların hepsini import etmem mi gerekiyor, yoksa sorunluları reddedebilir miyim?

C: Bu sizin tasarım kararınız. Doğru cevap yok, gerekçeli cevap var. README'de seçiminizi savunun.

S: Verinin doğal outlier'ları varsa bunlar hata mı, normal mi?

C: Bu sorunun cevabını sizin keşfetmeniz, sınıflandırmanız ve raporlamanız bekleniyor.

S: API endpoint çalışmıyorsa / erişim yoksa ne yapayım?

C: Bize bildirin. Mock bir endpoint (httpbin.org, webhook.site vb.) ile geliştirmeye devam edip, gerçek endpoint geldiğinde URL/key değiştirerek çalıştığını gösterebilirsiniz.

S: Hatalı kayıtları da API'ye göndereyim mi, yoksa sadece temiz olanları mı?

C: Sadece validate edilmiş ve onaylanmış kayıtları gönderin. Hatalı kayıtların hedef sisteme ulaşmaması, bu görevdeki en kritik gereksinimlerden biridir.

Başarılar dileriz.

Bu görev, sizin için olduğu kadar bizim için de bir mühendislik kültür uyumu testidir. Çözümünüz mükemmel olmasa bile düşünce şekliniz, dürüstlüğünüz ve dökümantasyonunuz mülakat sürecinin bir sonraki adımına geçmenizi sağlayabilir.